ARTIFICIAL INTELLIGENCE AND DEEP LEARNING

LAB ASSIGNMENT – 2

NAME – SUFIYA AFREEN

ENROLLMENT NO – 2403A51067

BATCH/BRANCH – CSE-03

DATE – 6-08-26

Lab/Exp 2: Building a Simple Linear Regression Model in PyTorch

Kaggle Dataset Link: https://www.kaggle.com/datasets/anninasimon/employee-salary-dataset

Tasks:

1. Load and preprocess the Salary Prediction dataset/example.
2. Build the required PyTorch model/program.
3. Train/execute the model.
4. Evaluate using suitable metrics.
5. Visualize results using plots where applicable.
6. Write a brief conclusion comparing observations.

CODE-

```
# Data handling
import pandas as pd
import numpy as np
import zipfile

# Visualization
import matplotlib.pyplot as plt

# Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# PyTorch
import torch
import torch.nn as nn
import torch.optim as optim
```

```
print(df.info())
print(df.describe())
print(df.isnull().sum())
```

[19] ✓ 0s

```
label_encoder = LabelEncoder()

for column in df.columns:
    if df[column].dtype == 'object':
        df[column] = label_encoder.fit_transform(df[column])
```

[20] ✓ 0s

```
X = df.iloc[:, 1:-1].values
y = np.log1p(df.iloc[:, -1].values) # Apply log transformation to the target variable
```

[26] ✓ 0s

```
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Initialize StandardScaler
scaler = StandardScaler()

# Fit on X_train and transform both X_train and X_test
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

X_train = torch.FloatTensor(X_train)
X_test = torch.FloatTensor(X_test)

y_train = torch.FloatTensor(y_train).view(-1,1)
y_test = torch.FloatTensor(y_test).view(-1,1)

class LinearRegressionModel(nn.Module):

    def __init__(self):
        super(LinearRegressionModel, self).__init__()
        # Reduced model complexity: single hidden layer with 8 neurons
        self.fc1 = nn.Linear(X_train.shape[1], 8) # First (and only) hidden layer
        self.relu = nn.ReLU() # Activation function
        self.dropout = nn.Dropout(0.2) # Dropout layer
        self.fc2 = nn.Linear(8, 1) # Output layer

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.fc2(x)
        return x
```

```python
model = LinearRegressionModel()
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.0001) # Reduced learning rate
epochs = 5000 # Increased number of epochs

losses = []

for epoch in range(epochs):

    predictions = model(X_train)

    loss = criterion(predictions, y_train)

    optimizer.zero_grad()

    loss.backward()

    optimizer.step()

    losses.append(loss.item())

    if (epoch+1)%500==0: # Print less frequently for higher epochs
        print(f"Epoch {epoch+1}, Loss = {loss.item():.4f}")
```

```python
model.eval()

with torch.no_grad():
    y_pred = model(X_test)
```

```python
model.eval()

with torch.no_grad():
    y_pred = model(X_test)

y_test_np_log = y_test.numpy()
y_pred_np_log = y_pred.numpy()

# Inverse transform predictions and actual values to original scale for metric calculation
y_test_np = np.expm1(y_test_np_log)
y_pred_np = np.expm1(y_pred_np_log)

# Check for NaN values in predictions (after inverse transform)
if np.isnan(y_pred_np).any():
    print("Error: Model predictions contain NaN values even after inverse transformation. This might indicate issues
    print("Cannot compute meaningful regression metrics with NaN predictions.")
else:
    mse = mean_squared_error(y_test_np, y_pred_np)
    mae = mean_absolute_error(y_test_np, y_pred_np)
    r2 = r2_score(y_test_np, y_pred_np)

    print("Mean Squared Error :", mse)
    print("Mean Absolute Error:", mae)
    print("R2 Score:", r2)

plt.figure(figsize=(8,5))
plt.plot(losses)
plt.title("Training Loss (Log Scale Target)")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.show()
```

OUTPUT-

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 35 entries, 0 to 34
Data columns (total 5 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   ID                35 non-null     int64
 1   Experience_Years  35 non-null     int64
 2   Age               35 non-null     int64
 3   Gender            35 non-null     int64
 4   Salary            35 non-null     int64
dtypes: int64(5)
memory usage: 1.5 KB
None
              ID  Experience_Years        Age     Gender        Salary
count  35.000000          35.00000  35.000000  35.000000  3.500000e+01
mean   18.000000           9.20000  35.485714   0.485714  2.059147e+06
std    10.246951           7.55295  14.643552   0.507093  3.170124e+06
min     1.000000           1.00000  17.000000   0.000000  3.000000e+03
25%     9.500000           2.50000  22.500000   0.000000  2.250000e+04
50%    18.000000           6.00000  29.000000   0.000000  2.500000e+05
75%    26.500000          15.00000  53.500000   1.000000  3.270000e+06
max    35.000000          27.00000  62.000000   1.000000  1.000000e+07
ID                 0
Experience_Years   0
Age                0
Gender             0
Salary             0
dtype: int64
```

```
Epoch 500, Loss = 151.0844
Epoch 1000, Loss = 143.3495
Epoch 1500, Loss = 133.7285
Epoch 2000, Loss = 124.1651
Epoch 2500, Loss = 119.0358
Epoch 3000, Loss = 109.1471
Epoch 3500, Loss = 91.8163
Epoch 4000, Loss = 79.5322
Epoch 4500, Loss = 63.6608
Epoch 5000, Loss = 64.8349
```

```
Mean Squared Error : 23626135552.0
Mean Absolute Error: 97009.0546875
R2 Score: -0.6606905460357666
```